

Module 7: Async JavaScript

Async JavaScript is used to perform tasks that take some time to complete, such as:

- Getting data from an API
- Reading data from a database
- Uploading or downloading files
- Using timers

Without Async JavaScript, the browser may freeze while waiting for these tasks to finish.

1. Promise

A **Promise** is an object that represents the future result of an asynchronous operation.

A Promise has three states:

1. **Pending** → Task is still running
 2. **Fulfilled (Resolved)** → Task completed successfully
 3. **Rejected** → Task failed
-

Syntax

```
let promise = new Promise((resolve, reject) => {  
  let success = true;  
  if(success){  
    resolve("Task Completed");  
  } else {  
    reject("Task Failed");  
  }  
});
```

Example 1: Student Result

```
let result = new Promise((resolve, reject) => {  
  let marks = 75;  
  if(marks >= 40){  
    resolve("Pass");  
  } else {  
    reject("Fail");  
  }  
});  
result  
.then((message) => {  
  console.log(message);  
})  
.catch((error) => {  
  console.log(error);  
});
```

Output

Pass

Example 2: Pizza Delivery

```
let pizzaOrder = new Promise((resolve) => {  
  
    setTimeout(() => {  
        resolve("Pizza Delivered");  
    }, 3000);  
  
});  
  
pizzaOrder.then((message) => {  
    console.log(message);  
});
```

Output (after 3 seconds)

Pizza Delivered

2. Async / Await

Async/Await makes Promise-based code easier to read and write.

async

The async keyword makes a function asynchronous.

```
async function greet() {  
    return "Hello";  
}
```

```
greet().then(console.log);
```

Output

Hello

await

The await keyword waits for a Promise to finish.

Example

```
function getData() {  
  
    return new Promise((resolve) => {  
  
        setTimeout(() => {  
            resolve("Data Loaded");  
        }, 2000);  
  
    });  
  
}  
  
async function showData() {  
  
    let result = await getData();  
  
    console.log(result);  
  
}
```

```
showData();
```

Output (after 2 seconds)

Data Loaded

Real-Life Example

```
function makeTea() {  
  
    return new Promise((resolve) => {  
  
        setTimeout(() => {  
            resolve("Tea Ready");  
        }, 3000);  
  
    });  
  
}  
  
async function serveTea() {  
  
    console.log("Making Tea...");  
  
    let tea = await makeTea();  
  
    console.log(tea);  
  
}
```

```
serveTea();
```

Output

Making Tea...

Tea Ready

3. JSON

JSON (JavaScript Object Notation) is a format used to store and exchange data.

JSON Example

```
{  
  "name": "Vicky",  
  "age": 25,  
  "city": "Udaipur"  
}
```

JavaScript Object

```
let student = {  
  name: "Vicky",  
  age: 25,  
  city: "Udaipur"  
};
```

```
console.log(student.name);
```

Output

Vicky

Convert Object to JSON

JSON.stringify()

```
let student = {  
  name: "Vicky",  
  age: 25  
};
```

```
let jsonData = JSON.stringify(student);
```

```
console.log(jsonData);
```

Output

```
{"name":"Vicky","age":25}
```

Convert JSON to Object

JSON.parse()

```
let jsonData = '{"name":"Vicky","age":25}';
```

```
let obj = JSON.parse(jsonData);
```

```
console.log(obj.name);
```

Output

Vicky

4. Fetch API

The **Fetch API** is used to get data from a server or API.

Syntax

```
fetch(url)
.then(response => response.json())
.then(data => console.log(data));
```

Example

```
fetch("https://jsonplaceholder.typicode.com/users")
.then(response => response.json())
.then(data => console.log(data))
.catch(error => console.log(error));
```

Fetch Using Async/Await

```
async function getUsers() {

    let response =
        await
        fetch("https://jsonplaceholder.typicode.com/users");

    let data =
        await response.json();
```



```
    console.log(data);  
  
}
```

```
getUsers();
```

5. API Calls

API (Application Programming Interface) allows two applications to communicate with each other.

Flow

Browser → API → Server

Server → API → Browser

Example API Call

```
fetch("https://jsonplaceholder.typicode.com/posts/1")  
.then(response => response.json())  
.then(data => console.log(data));
```

Sample Output

```
{  
  userId: 1,  
  id: 1,  
  title: "...",  
  body: "..."  
}
```

Practical Example 1: Load User Data

```
<!DOCTYPE html>
<html>
<body>

<button onclick="loadUser()">
Load User
</button>

<div id="result"></div>

<script>

async function loadUser() {

    let response =
        await
fetch("https://jsonplaceholder.typicode.com/users/1");

    let user =
        await response.json();

    document.getElementById("result").innerHTML =
        `
        Name: ${user.name}<br>
        Email: ${user.email}
        `;

}
```

```
</script>
```

```
</body>
```

```
</html>
```

Practical Example 2: Random Joke Generator

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<button onclick="getJoke()">
```

```
Get Joke
```

```
</button>
```

```
<h2 id="joke"></h2>
```

```
<script>
```

```
async function getJoke() {
```

```
    let response =
```

```
        await fetch("https://official-joke-  
api.appspot.com/random_joke");
```

```
    let data =
```

```
        await response.json();
```

```
    document.getElementById("joke").innerHTML =
```

```
        data.setup + "<br>" + data.punchline;
```

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```

Promise vs Async/Await

Promise	Async/Await
Uses .then()	Uses await
More code	Less code
Can be harder to read	Easy to read
Uses chaining	Looks like normal code

Promise

```
fetch(url)
  .then(response => response.json())
  .then(data => console.log(data));
```

Async/Await

```
async function getData() {

  let response = await fetch(url);

  let data = await response.json();

  console.log(data);
```

}

Interview Questions

1. What is a Promise?

A Promise is an object that represents the future result of an asynchronous task.

2. What are the three states of a Promise?

- Pending
- Fulfilled
- Rejected

3. What does async do?

It makes a function asynchronous and returns a Promise.

4. Where can await be used?

Inside an async function only.

5. What does JSON.stringify() do?

It converts a JavaScript object into a JSON string.

6. What does JSON.parse() do?

It converts a JSON string into a JavaScript object.

7. What is the Fetch API?

A modern way to get data from a server or API.

8. What is an API?

A tool that allows two applications to communicate with each other.

Easy Real-Life Example

Promise = Ordering Pizza

Order Pizza → Pending

Pizza Delivered → Fulfilled

Restaurant Closed → Rejected

Async/Await = Waiting for Pizza

```
let pizza = await orderPizza();
```

```
console.log(pizza);
```

You place an order and wait until the pizza arrives. After it arrives, you use it.

Practical Examples of Async JavaScript (Beginner to Intermediate)

These examples are useful for students because they are based on real-life web applications.

1. Loading Message After 3 Seconds (Promise)

Use Case:

Showing a loading screen before displaying content.

```
let loading = new Promise((resolve) => {  
  
    setTimeout(() => {  
        resolve("Website Loaded Successfully!");  
    }, 3000);  
  
});  
  
loading.then((message) => {  
    console.log(message);  
});
```

Output

Website Loaded Successfully!

2. Online Exam Result Checker

Use Case:

Checking whether a student has passed or failed.

```
function checkResult(marks){  
  
    return new Promise((resolve,reject)=>{  
  
        if(marks >= 40){
```

```
        resolve("Congratulations! You Passed");
    }
    else{
        reject("Sorry! You Failed");
    }

});

}

checkResult(65)
.then(result => console.log(result))
.catch(error => console.log(error));
```

3. ATM Balance Check

Use Case:

Banking Applications

```
function checkBalance(){

    return new Promise((resolve)=>{

        setTimeout(()=>{
            resolve("Available Balance: ₹25,000");
        },2000);

    });

}
```



```
async function showBalance(){  
  
    let balance = await checkBalance();  
  
    console.log(balance);  
  
}  
  
showBalance();
```

4. Student Data Viewer (JSON)

Use Case:

School Management System

```
let student = `{  
    "name":"Vicky",  
    "course":"BCA",  
    "marks":88  
}`;  
  
let data = JSON.parse(student);  
  
console.log(data.name);  
console.log(data.course);  
console.log(data.marks);
```

Output

Vicky

BCA

88

5. To-Do List Data in JSON

```
let tasks = [  
  {  
    task:"Study JavaScript",  
    completed:false  
  },  
  {  
    task:"Complete Assignment",  
    completed:true  
  }  
];  
  
let jsonData = JSON.stringify(tasks);  
  
console.log(jsonData);
```

6. Random User Generator (Fetch API)

Use Case:

Getting user information from an API.

```
<!DOCTYPE html>  
<html>  
<body>
```

```
<button onclick="getUser()">
```

```
Get User
```

```
</button>
```

```
<h2 id="result"></h2>
```

```
<script>
```

```
async function getUser(){
```

```
    let response =
```

```
    await fetch("https://randomuser.me/api/");
```

```
    let data =
```

```
    await response.json();
```

```
    document.getElementById("result").innerHTML =
```

```
    data.results[0].name.first;
```

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```

7. Country Information App

Use Case:

Travel Website

```
<!DOCTYPE html>
<html>
<body>

<button onclick="getCountry()">
Country Info
</button>

<div id="result"></div>

<script>

async function getCountry(){

    let response =
    await fetch("https://restcountries.com/v3.1/name/india");

    let data =
    await response.json();

    document.getElementById("result").innerHTML =
    `
    Country: ${data[0].name.common}<br>
    Capital: ${data[0].capital}
    `;

}

</script>
```

```
</body>  
</html>
```

8. Live Digital Clock

Use Case:

Dashboard Applications

```
<!DOCTYPE html>  
<html>  
<body>
```

```
<h1 id="clock"></h1>
```

```
<script>
```

```
function showTime(){
```

```
    let now = new Date();
```

```
    document.getElementById("clock").innerHTML =  
    now.toLocaleTimeString();
```

```
}
```

```
setInterval(showTime,1000);
```

```
</script>
```

```
</body>  
</html>
```

9. Weather App (Async/Await)

Use Case:

Weather Websites

```
async function getWeather(){  
  
    let city = "Delhi";  
  
    let response =  
    await fetch(  
    `https://api.openweathermap.org/data/...`  
    );  
  
    let data =  
    await response.json();  
  
    console.log(data);  
  
}
```

10. GitHub User Search

Use Case:

Developer Portfolio Websites

```
<!DOCTYPE html>
<html>
<body>

<input type="text" id="username">
<button onclick="searchUser()">
Search
</button>

<div id="result"></div>

<script>

async function searchUser(){

    let user =
    document.getElementById("username").value;

    let response =
    await fetch(`https://api.github.com/users/${user}`);

    let data =
    await response.json();

    document.getElementById("result").innerHTML =
    `
    Name: ${data.name}<br>
    Followers: ${data.followers}
    `;

}
```

```
</script>
```

```
</body>
```

```
</html>
```

11. Joke Generator

Use Case:

Entertainment Websites

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<button onclick="getJoke()">
```

New Joke

```
</button>
```

```
<p id="joke"></p>
```

```
<script>
```

```
async function getJoke(){
```

```
  let response =
```

```
  await fetch(
```

```
    "https://official-joke-api.appspot.com/random_joke"
```

```
  );
```



```
let joke =  
  await response.json();  
  
  document.getElementById("joke").innerHTML =  
    joke.setup + "<br>" + joke.punchline;  
  
}  
  
</script>  
  
</body>  
</html>
```

12. Product Loader (E-commerce)

Use Case:

Amazon / Flipkart Type Websites

```
async function loadProducts(){  
  
  let response =  
    await fetch("https://fakestoreapi.com/products");  
  
  let products =  
    await response.json();  
  
  products.forEach(product => {  
  
    console.log(product.title);  
  
  })  
}
```

```
});  
  
}  
  
loadProducts();
```

13. Movie Search App

Use Case:

Netflix / IMDb Style Applications

```
async function searchMovie(){  
  
    let response =  
    await fetch(  
  
"https://www.omdbapi.com/?apikey=YOUR_KEY&t=Avatar"  
    );  
  
    let movie =  
    await response.json();  
  
    console.log(movie.Title);  
    console.log(movie.Year);  
  
}
```

14. Simulating Online Shopping Checkout

Promise Example

```
function placeOrder(){  
  
    return new Promise((resolve)=>{  
  
        setTimeout(()=>{  
            resolve("Order Placed Successfully");  
        },3000);  
  
    });  
  
}  
  
async function checkout(){  
  
    console.log("Processing Order...");  
  
    let result =  
    await placeOrder();  
  
    console.log(result);  
  
}  
  
checkout();
```

Output

Processing Order...
Order Placed Successfully

15. File Download Simulation

Promise + Async/Await

```
function downloadFile(){

    return new Promise((resolve)=>{

        setTimeout(()=>{
            resolve("File Downloaded");
        },5000);

    });

}

async function startDownload(){

    console.log("Downloading...");

    let result =
    await downloadFile();

    console.log(result);

}

startDownload();
```